DOCUMENTAÇÃO TÉCNICA

Lógica de programação e Java no CRUD administrativo

Projeto Mottainai

Responsável	João Victor Rodrigues
Disciplina	Lógica de Programação (LPR) / JAVA
Equipe / colaboradores	João Victor Rodrigues, Pedro Henrique Ângelo e Miguel Quadros Araújo
Versão	1.0 | 29/10/2026

Finalidade

Este documento registra a aplicação de lógica de programação e Java no CRUD administrativo do Projeto Mottainai. Apresenta métodos, estruturas de decisão e repetição, comentários, validações e tratamento de exceções utilizados para receber dados, executar regras e responder ao usuário.

1. Contexto e objetivo

Lógica de Programação contribui para transformar entradas dos formulários em operações válidas sobre os dados administrativos. O código Java verifica campos obrigatórios, calcula dígitos do CPF, controla vínculos, filtra listas e trata situações que impedem a conclusão de uma operação.

A documentação de LPR/JAVA abrange comentários, exceções e validações. Ela complementa a documentação de POO ao explicar os algoritmos e as decisões executadas dentro das classes do CRUD.

Responsabilidade individual

João Victor Rodrigues é responsável por implementar as classes do sistema, desenvolver Model, DTO, Service e Controller, validar as entradas, tratar exceções e documentar os testes e as evidências de entrega.

Pedro Henrique Ângelo é responsável pela integração com PostgreSQL via JDBC e pelas estruturas de acesso a dados em Java puro. Miguel Quadros Araújo é responsável pela revisão dos códigos e pela integração das requisições HTTP e das páginas JSP com o backend Spring MVC.

Decisão técnica

A linguagem adotada é Java 21. O projeto usa Spring Boot 4.0.0, Gradle, Bean Validation, JPA e JDBC. A lógica aparece em métodos próprios, como calculateCheckDigit, matchesStatus, addPage e deletionTarget, além das validações declarativas do framework.

Relação com as demandas do projeto

Disciplina	Exigência	Como esta entrega contribui
LPR / JAVA	Comentários, exceções e validações.	Regras e algoritmos concretos nos arquivos Java.
POO	Classes e responsabilidades.	Mesmos DTOs, Services e Controllers do CRUD.
Banco de Dados 1	Entradas e relacionamentos válidos.	Verificações antes da persistência.
MPTI e entrega geral	Organização e evidências.	Requisito relacionado a arquivo e cenário de teste.

2. Especificações da disciplina

Tecnologia / ferramenta	Uso no projeto
Java 21	Tipos, operadores, condições, laços, métodos e coleções.
Bean Validation	Restrições nos DTOs e acionamento por @Valid.
ConstraintValidator	Contrato implementado por CpfValidator.
Spring MVC / MockMvc	Entrada HTTP e testes de resposta.
Stream / Optional	Filtros, conversões e ausência de registro.
JPA / JDBC	Consultas e gravação após verificações.
JUnit / Mockito	Cenários de sucesso, erro e regras do domínio.

3. Parte técnica

O processamento começa nos Controllers. O DTO recebe os valores e as validações de entrada são verificadas. O Service consulta registros, avalia condições e decide se pode salvar. Uma exceção ou mensagem de validação interrompe o caminho de sucesso e orienta a resposta ao usuário.

Componente / artefato	Responsabilidade	Evidência no código
Comentários	Explicar intenção, regra e limite.	CpfValidator e DataEncryptionService.
Validação de entrada	Rejeitar campos e formatos inválidos.	RequestDTOs e ValidCPF.
Regras do domínio	Impedir operações incompatíveis.	EmployeeService e AdministrationService.
Tratamento de erros	Responder com status ou mensagem.	GlobalExceptionHandler e ViewControllers.
Algoritmos de lista	Filtrar, ordenar e paginar.	ViewFilterUtils e ViewPaginationUtils.

Comentários no código

Os comentários explicam por que uma verificação existe. CpfValidator informa que CPF vazio é responsabilidade de @NotBlank e que sequências repetidas devem ser rejeitadas. EmployeeService distingue exclusão lógica de remoção física e explica a preservação do nome cifrado.

DataEncryptionService documenta chave, IV, tag e formato armazenado. AdministrationService explica que o ID do solicitante vem da sessão. Esses comentários relacionam a intenção da regra com as verificações executadas no código.

Tipos variáveis e métodos

O código utiliza String para textos, Integer e int para identificadores e cálculos, boolean para situação, LocalDate para datas e LocalDateTime para registros de tempo. No CPF, int[] armazena os dígitos. List reúne resultados e Optional indica que uma consulta pode não localizar um registro.

Métodos menores dividem a tarefa: isValid prepara e verifica o CPF; validateCheckDigits compara os verificadores; calculateCheckDigit executa a soma ponderada. A separação permite compreender entradas e saídas e reutilizar o cálculo para os dois dígitos.

Decisões e repetição

if/else aparece na rejeição de dados vazios, na verificação de registros excluídos e no bloqueio de autoexclusão. O operador ternário escolhe o dígito do CPF conforme o resto da divisão. O laço for percorre os dígitos com pesos decrescentes. Streams filtram, ordenam e transformam coleções em DTOs.

Validação de CPF

CpfValidator recebe String. Se for nula ou vazia, retorna true e deixa a obrigatoriedade para outra anotação. Remove os caracteres não numéricos, exige 11 dígitos e rejeita sequências com dígitos iguais. A verificação matemática não comprova a existência de CPF cadastrado.

O primeiro verificador multiplica os nove primeiros dígitos por pesos de 10 a 2 e acumula a soma. Calcula o resto por 11: se for menor que 2, o dígito é 0; caso contrário, é 11 menos o resto. O segundo cálculo usa dez dígitos e pesos de 11 a 2. Os resultados são comparados às duas últimas posições.

Figura 1 – Etapas do algoritmo de CPF.

Entrada → remover formatação → verificar 11 dígitos → rejeitar repetição → calcular primeiro dígito → calcular segundo dígito → retornar válido ou inválido.

Validações de entrada e de negócio

Os DTOs utilizam anotações como @NotBlank e @Email conforme o campo, acionadas nos pontos com @Valid. Regras que dependem do banco ficam nos Services: EmployeeService rejeita loja ou cargo excluídos e AdministrationService verifica e-mail duplicado.

deletionTarget rejeita sessão sem ID e compara o alvo da exclusão ao administrador autenticado. Se forem iguais, lança BusinessRuleException. O controle é aplicado antes da chamada ao DAO e não depende apenas do botão exibido na interface.

Busca e paginação

ViewFilterUtils normaliza textos para comparação e aplica busca e situação ativo/inativo. Situação vazia ou não reconhecida mantém a listagem. Os ViewControllers filtram os dados antes da paginação.

ViewPaginationUtils define PAGE_SIZE=6. Com total=0, mantém uma página; nos outros casos, calcula (total−1)/6+1 com divisão inteira. Ajusta a página entre 1 e o limite e calcula start=(current−1)×6 e end=min(start+6,total). A tabela recebe subList(start,end).

Exemplo: 13 registros produzem três páginas. Na página 3, start=12 e end=13; a tabela recebe um item. Com lista vazia, página=1 e índices inicial e final apresentados=0. O algoritmo opera na lista já carregada, sem limitar a consulta SQL do CRUD.

Tratamento de exceções

ResourceNotFoundException representa ausência; DuplicateResourceException, duplicidade; BusinessRuleException, uma operação proibida. GlobalExceptionHandler atua nos Controllers REST e converte falhas em ErrorResponseDTO com status, descrição e caminho da requisição.

Falha / condição	Resposta implementada
Bean Validation ou argumento inválido	HTTP 400 com indicação do problema de entrada.
ResourceNotFoundException	HTTP 404 para recurso não encontrado.
Duplicidade ou regra de negócio	HTTP 409 para conflito ou operação proibida.
Falha técnica inesperada	HTTP 500 com mensagem genérica.
Erro de protocolo reconhecido pelo Spring	Preserva status correspondente, como 405 ou 415.

Nas telas, ViewControllers usam BindingResult e mensagens flash. No login, AuthenticationController trata DataAccessException e informa indisponibilidade sem expor a conexão. DataEncryptionService usa try/catch para converter erros de cifra e de formato em IllegalStateException.

Lógica na proteção de dados

AES utiliza uma sequência com arrays de bytes e conversões: gerar IV, converter String em UTF-8, cifrar, concatenar e codificar em Base64. Na volta, verifica tamanho, separa IV e conteúdo e valida a tag. BCrypt utiliza matches para comparar a senha com o hash armazenado.

DataAnonymizationService gera ANON_ com UUID e fornece data substituta. anonymizeNumericId calcula os zeros necessários para juntar prefixo e ID em tamanho fixo; se o identificador não couber, lança IllegalArgumentException. Esse valor não é uma cifra e mantém o ID interno reconhecível.

Implementação entregue

A implementação entregue contém comentários explicativos, validador de CPF, restrições de entrada, exceções próprias, tratamento HTTP, busca, paginação e testes relacionados. As seções seguintes identificam os requisitos, os cenários e seus arquivos de referência.

4. Requisitos e entregáveis

ID	Requisito / entregável	Local no repositório	Status
R1	Comentários de regras e decisões.	validation/, service/ e security/	Implementado

ID	Requisito / entregável	Local no repositório	Status
R2	Validação declarativa e CPF.	dto/ e validation/	Implementado
R3	Decisões, laços e métodos.	CpfValidator e util/	Implementado
R4	Exceções e respostas de erro.	exception/ e controller/	Implementado
R5	Cenários de teste.	src/test/java/	Disponíveis

5. Validação e testes

Critérios de validação

- Campos obrigatórios e formatos inválidos devem impedir cadastro nos pontos validados.
- CPF deve respeitar tamanho, repetição e cálculo dos verificadores.
- Ausência e operações proibidas devem produzir respostas coerentes.
- Paginação deve tratar lista vazia, última página e página fora do limite.
- Comentários e evidências devem explicar a regra sem credenciais ou dados reais.

Casos de teste

ID	Cenário	Resultado esperado	Evidência de referência
T1	CPF incorreto ou repetido.	Retornar false.	Implementado em CpfValidator; sem registro específico de execução.
T2	CPF nulo em campo obrigatório.	Validador delega; @NotBlank rejeita.	Implementado no validador e no DTO; sem registro específico de execução.
T3	Busca e situação.	Retornar itens compatíveis.	ViewFilterRegressionTest.
T4	Lista vazia ou página fora do limite.	Ajustar página e índices.	ViewPaginationUtilsTest.
T5	Entrada HTTP inválida ou ausência.	Retornar 400 ou 404.	HttpErrorReviewTest.
T6	Operação proibida.	Bloquear com conflito.	MvcBusinessErrorReviewTest e AdministrationDeletionTest.

Registros de execução dos testes

O relatório build/manual/test-summary.txt contém 55 testes encontrados, 54 executados e aprovados, 1 ignorado e nenhuma falha. O arquivo build/documentation-final-tests.log registra uma execução separada com 28 testes aprovados. Esses números descrevem execuções distintas, sem total acumulado. Os códigos dos testes estão em src/test/java/com/mottainai/crudinicial/.

O escopo do resumo local inclui testes de regras e requisições com recursos simulados. O fluxo FinalFrontendLiveIntegrationTest depende de MOTTAINAI_LIVE_TESTS=true e de PostgreSQL configurado; sua execução não está comprovada nesse resumo. CrudInicialApplicationTests depende do banco e não integra a seleção de scripts/LocalTestRunner.java.

6. Configuração e execução

Passo	Ação
1	Instalar JDK 21 e abrir o projeto Gradle no IntelliJ IDEA ou no terminal.
2	Preparar PostgreSQL com tabelas compatíveis com os modelos e registros técnicos de anonimização. Hibernate usa ddl-auto=validate.
3	Definir variáveis do banco e a chave AES no ambiente. O perfil dev já está indicado em application.properties.
4	Executar .\gradlew.bat bootRun no Windows. Para testes, executar .\gradlew.bat test com o ambiente configurado.
5	Abrir http://localhost:8080/login e autenticar uma conta Administration previamente cadastrada.

Variável / configuração	Finalidade	Exemplo seguro
DB_HOST / DB_PORT	Servidor e porta PostgreSQL.	Definidos no ambiente
DB_NAME	Nome do banco.	Definido no ambiente
DB_USER / DB_PASSWORD	Credenciais da conexão.	Definidas no ambiente
MOTTAINAI_AES_KEY	Base64 de chave AES de 32 bytes.	Definida no ambiente
SPRING_PROFILES_ACTIVE	Seleção do perfil Spring.	dev
MOTTAINAI_LIVE_TESTS	Habilita testes com banco real.	true no ambiente de teste

A conexão utiliza sslmode=require. A configuração valida o schema existente, sem criar ou alterar tabelas. A execução pressupõe tabelas compatíveis com os modelos e registros técnicos de anonimização previamente cadastrados. Os dados cifrados exigem a mesma chave para recuperação. As credenciais do banco e a chave AES são fornecidas pelo ambiente.

7. Evidências de entrega

Os caminhos relativos das classes Java abaixo partem de src/main/java/com/mottainai/crudinicial/. Os testes ficam em src/test/java/com/mottainai/crudinicial/ e os relatórios de execução estão em build/.

Evidência	Link ou local	Status
Comentários	validation/CpfValidator.java e security/DataEncryptionService.java	Presentes
Exceções e respostas	exception/GlobalExceptionHandler.java	Presente
Validações e algoritmos	dto/, validation/ e util/	Presentes
Testes e resultado local	src/test/java/ e build/manual/test-summary.txt	Disponíveis

8. Conclusão

A implementação aplica lógica de programação em métodos Java que recebem entradas, avaliam condições, percorrem dados, calculam verificadores, filtram listas e tratam falhas. Comentários, validações e exceções estão relacionados aos arquivos e às regras do CRUD administrativo.

As validações separam formato e obrigatoriedade das regras que dependem do banco. Os algoritmos de CPF, busca e paginação demonstram o uso de operadores, decisões, repetição e métodos. O tratamento de exceções converte falhas em mensagens e respostas HTTP coerentes com o processamento da aplicação.

Histórico de versões

Versão	Data	Autor	Alterações
1.0	29/10/2026	Equipe LPR/JAVA	Lógica Java, comentários, validações, exceções e testes.

Referência de demanda

Projeto Interdisciplinar – Solicitações por Disciplina, 2026. Itens: Java, comentários, validação, exceções e integração com o CRUD.